Experiment 1 - HDL Clock Generation and Frequency Divider

Ali Dehghani,
810103414

***Abstract*— This report explores the Verilog HDL modeling and FPGA implementation of clock generation and frequency division circuits. It includes behavioral simulations of an LM555 timer and a ring oscillator, and details the schematic design of a divide-by-113 synchronous counter with a T flip-flop to accurately measure high-frequency signals using a digital display module.**

Keywords—Verilog HDL, FPGA, Frequency Divider, Synchronous Counter, Ring Oscillator, ModelSim Simulation.

I. Introduction

This experiment transitions from analog timing analysis to digital system design using Verilog HDL and FPGA implementation. The study involves modeling previously explored clock generators—an LM555 timer and a ring oscillator—using behavioral modeling and delay statements. Furthermore, it focuses on constructing a practical frequency divider system utilizing 74-series logic gates in Intel Quartus. By integrating a divide-by-113 synchronous counter, a T flip-flop to ensure a 50% duty cycle, and a display measurement module, the experiment provides a comprehensive framework for analyzing and measuring high-frequency digital signals.

II. Clock Generation using HDL

A. *Behavioral Modeling using RTL Components*

1) *Block Diagram and Explain Hardware:*

Clk and reset inputs and the pulse output.Two counter blocks.Comparator blocks to evaluate counter values against constant thresholds to generate the required enable and reset signals. Logic gates (AND/OR) to manage the control signals.A flip-flop or register the state of the pulse output.

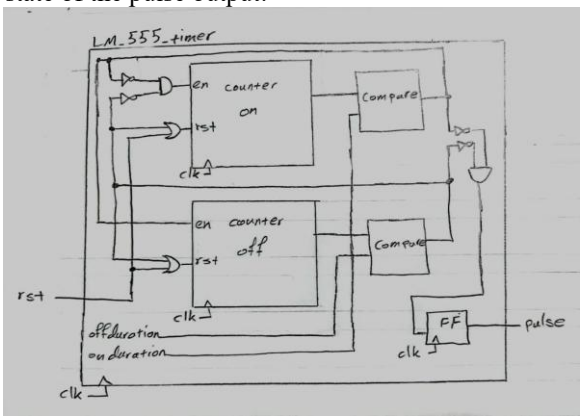


Fig. 1 Block diagram

2) *Explain Functionality:*

In the pre-processing phase, the High and Low durations are calculated based on the standard 555 timer formulas:

$$T_{on} = 0.693 \times (R_1 + R_2) \times C$$
$$T_{off} = 0.693 \times R_2 \times C$$

Initialization: Upon system startup, the count_on counter increments by one on every rising edge of the clock signal. During this phase, the pulse output remains High.

On-Period Completion: Once count_on reaches the predefined onduration threshold, the counter stops, and the pulse output transitions to Low.

Off-Period Execution: Subsequently, the count_off counter begins incrementing until it reaches the offduration threshold.

Cycle Reset: Upon completion of the off-period, both counters are reset, and the cycle repeats automatically from the beginning.

3) *Waveform:*

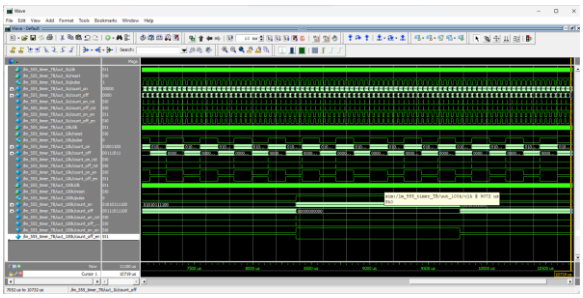


Fig. 2 LM555Timer waveform

4) *Measure Duty Cycles:*

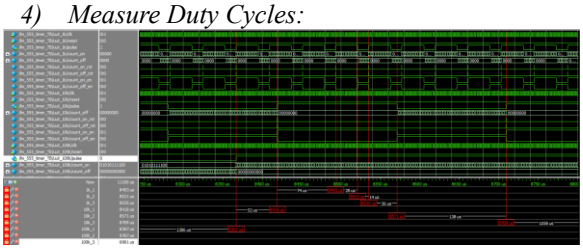


Fig. 3 Add cursor to waveform

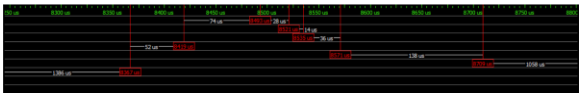


Fig. 4 Measure duty cycle

$$DutyCycle = \frac{T_{on}}{T_{on} + T_{off}} \times 100$$

$$DutyCycle_{1k} = \frac{28\mu s}{28\mu s + 14\mu s} \times 100 = 66\%$$

$$DutyCycle_{10k} = \frac{152\mu s}{152\mu s + 138\mu s} \times 100 = 52.41\%$$

$$DutyCycle_{100k} = \frac{1400\mu s}{1400\mu s + 1386\mu s} \times 100 = 50.25\%$$

5) Compare Results:

As we can see result of two sections completely matched.

The agreement between the ModelSim simulation results and the LTspice results is due to the fact that the digital model written in Verilog was designed directly from the same mathematical relationships that govern the charge and discharge of the capacitor in the LM555 timer.

In a 555 astable timer, the ON and OFF times follow these well-known relations:

$$T_{on} = 0.693\,(R_1 + R_2)\,C$$
$$T_{off} = 0.693\,R_2\,C$$

Accordingly, the duty cycle is:

$$D = \frac{T_{on}}{T_{on} + T_{off}} = \frac{R_1 + R_2}{R_1 + 2R_2}$$

From a mathematical standpoint, when R_2becomes much larger than R_1(i.e., $R_2 \gg R_1$), the R_1term becomes negligible in both the numerator and the denominator, and the expression approaches:

$$D \approx \frac{R_2}{2R_2} = 50\%$$

Because the Verilog code is implemented as a behavioral model that reproduces these same proportional relationships ,its duty-cycle results closely match those of the analog 555 circuit simulated in LTspice.

However, despite the overall agreement in behavior, the Verilog results are essentially ideal, since they rely on discrete clock-cycle counting with no device-level non-idealities. In contrast, LTspice includes more realistic effects such as comparator and flip-flop propagation delays inside the 555. At higher frequencies, these non-idealities can cause small deviations (often on the order of 1–2%) from the ideal theoretical formulas.

B. *Using delay statement*

1) Verilog Description and Test Bench:

For ring oscillator I used generate for invertors and parameter for number of them and delay of each one. And in test bench in the beginning force the first wire to zero(Avoid X value) and set 107ns delay(based on previous experiment) and 3 inverter.

Project of this part is inside 2-B folder.

2) Measure Frequency:

As we can see from Fig. 5 we have 4 wire that the last and the first one are connected(3 invertors).

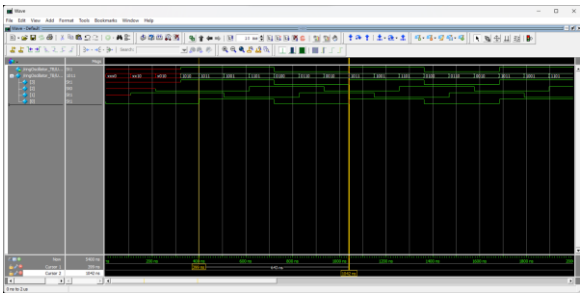


Fig. 5 Ring oscillator waveform

$$T = 643ns$$
$$f = \frac{1}{T} = \frac{1}{643 \times 10^{-9}\ s} \approx 1.557\ MHz$$

The frequency of this experiment and the previous are the same.

III. FPGA DESIGN

A. *Synchronous Counter as a Frequency Divider*

At first I created a block diagram with 74191 IC but there was a problem that when the frequency divider reach to 191 set RCON to 0 so it load again and doesn't reach to 255, and then I used 74193 IC and that wok correctly.

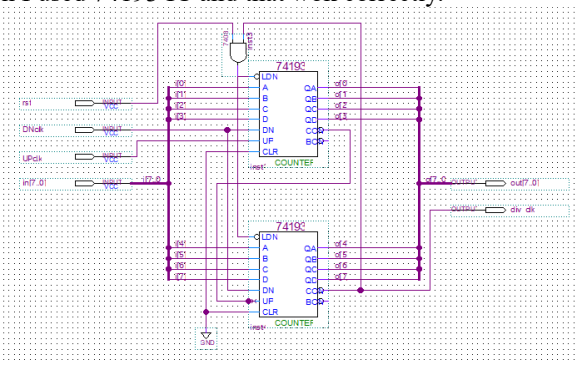


Fig. 5 Frequency divider block diagram

I used 8 bit bus for input and output and some more signals.

In testbench for construct a divide by 113 we have to set load value as:

$$\text{Initial value} = \text{Maximum value} - \text{Modulus} = 255 - 113 = 142$$

Also DNclk should be 1 always(because we only want to count up)Another important point is because of using delays(.sdo file) ring oscillator frequency(from previous part) is so high for that so I increased the delay of a single invertor.

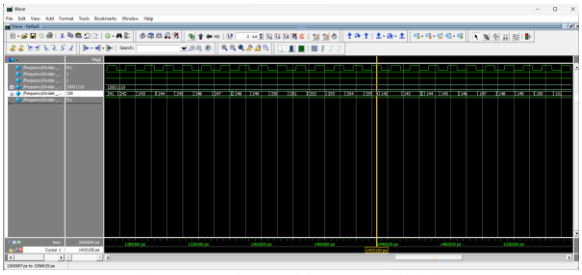


Fig. 6 Waveform of frequency divider

As we can see it goes up tp 255 and then load to 142.

Based on the simulation waveforms, the input clock generated by the ring oscillator has a period of $T_{in} \approx 12$ ns($f_{in} \approx 83.3$MHz):

$$T_{in} = 2 \times N \times delay = 2 \times 3 \times 2000 = 12000ps = 12ns$$

The counter correctly increments from 142 to 255. Upon reaching 255, the MXMN/RCON output generates a pulse, and the counter wraps back to 142.

By measuring the time difference between two consecutive output pulses on the waveform, the output period is observed to be exactly $113 \times T_{in}$. Therefore, the output frequency is successfully verified to be $f_{out} = \frac{f_{in}}{113}$, validating the functionality of the designed frequency divider.

$$T_{out} = 113 \times T_{in} = 113 \times 12ns = 1356ns$$

$$f_{out} = \frac{1}{T_{out}} = \frac{1}{1356 \times 10^{-9}} = 0.74MH$$

B. *T Flip-Flop*

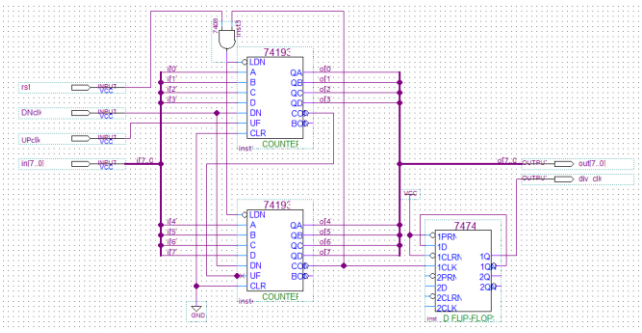


Fig. 7 TFF circuit block diagram

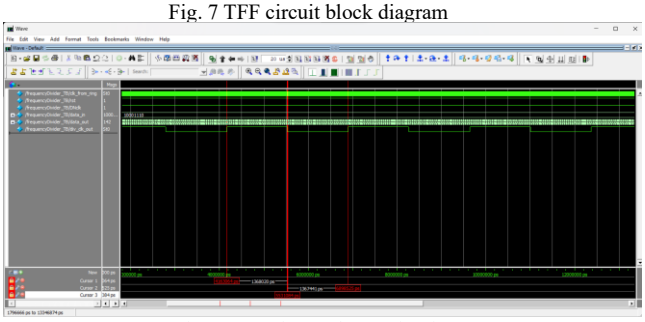


Fig. 8 Waveform of TFF circuit

As we can see from the cursors duty cycle is 50%.

C. *Display module in FPGA*

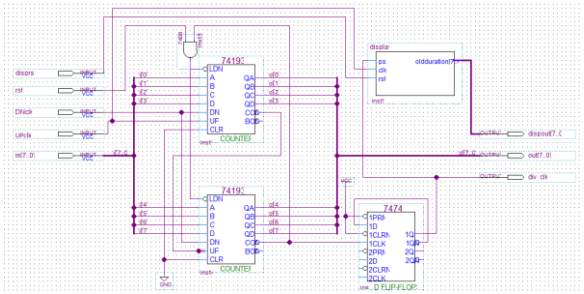


Fig. 9 Complete circuit block diagram

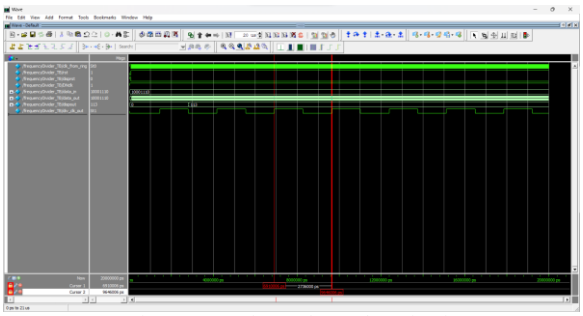


Fig. 10 Wave form of complete circuit

As we can see after the first time that div_clk_out becomes 1 dispout put that is output of display block becomes 113 and this is exactly the number we set for frequency divider to divide(num of clk that div_clk_out is 1).

Now we can calculate frequency of ring oscillator with:

$$T_{divided} = 2736000ps = 2736ns$$

$$f_{divided} = \frac{1}{T_{divided}} = \frac{1}{2736ns} \approx 0.36MH$$

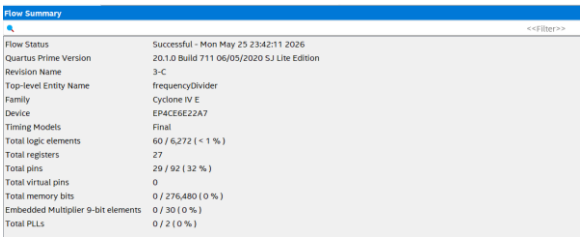
$$f_{ring} = f_{divided} \times (duration \times 2)$$
$$= 0.36MH \times (113 \times 2) = 41.36MH$$

That ×2 in the last equation is because of TFF we know that it's duty cycle is 50% then it's output equal to 1 half the time and for the frequency we have to multiply that with 2.

We can see the frequency that we calculated and the frequency of previous part are the same.

As I said at the fist of this session I changed delay of an invertor from 107 to 2000 because timing of all project is ps and 107ps delay is so low, then the frequency of ring oscillator is so high that circuit doesn't work si I change that to a reasonable value.

IV. APPENDICES



Flow Summary	
Flow Status	Successful - Mon May 25 23:42:11 2026
Quartus Prime Version	20.1.0 Build 711 06/05/2020 SJ Lite Edition
Revision Name	3-C
Top-level Entity Name	frequencyDivider
Family	Cyclone IV E
Device	EP4CE6E22A7
Timing Models	Final
Total logic elements	60 / 6,272 (< 1 %)
Total registers	27
Total pins	29 / 92 (32 %)
Total virtual pins	0
Total memory bits	0 / 276,480 (0 %)
Embedded Multiplier 9-bit elements	0 / 30 (0 %)
Total PLLs	0 / 2 (0 %)

Fig. 11 Flow summery of complete circuit

The design is highly efficient in terms of hardware usage. It utilizes only 60 out of 6,272 Total Logic Elements (<1%). Furthermore, the circuit incorporates 27 registers and uses 29 out of the 92 available pins (32%). No memory bits or PLLs were required for this implementation.